Basic Scale Document

Service Mobility Dashboard

Full-Stack Final Project

1. Purpose

This document explains how the current Service Mobility Dashboard supports basic scale, which parts of the implementation already reduce unnecessary work, where bottlenecks may appear as the dataset and number of dispatchers grow, and which improvements would be appropriate at a larger production scale.

The current system is an internal operational dashboard rather than an internet-scale consumer application. The scale discussion therefore focuses on realistic growth from a moderate fleet and dispatcher team to larger daily task volumes and higher concurrent usage. The analysis is independent of the organization's upstream ERP or operational source system; the current application consumes a normalized service-task dataset from Supabase. No production load benchmark is claimed in this document.

2. Expected Operating Scale

The current architecture is intended to remain practical for an internal organization with a moderate fleet and a moderate number of concurrent dispatchers. The important scaling dimensions are the number of historical service_tasks rows, the number of tasks scheduled per day, the number of vehicles that require route optimization, and the number of dispatchers opening the same workday at the same time.

The architecture is deliberately optimized around the operational workday: the main dashboard reads one selected date, while the recommendation flow reads only a four-day future window. If the product were expanded to thousands of daily tasks, very large fleets, multiple organizations, or heavy concurrent usage, additional optimizations would be required.

Upstream ingestion or synchronization is treated as a separate integration concern. Scaling the dashboard therefore does not require coupling its query, map, recommendation, or routing logic to a specific ERP vendor (for example, SAP).

3. Current Scale-Supporting Architecture

The current implementation separates server-side data access from client-side interaction.

Server-side work

- Authentication verification through Supabase.
- Date-scoped service-task retrieval.
- Four-day recommendation candidate retrieval.
- Server-side validation of client-controlled recommendation coordinates.

Client-side work

- Interactive Mapbox map rendering.
- Vehicle grouping and visibility/focus state.
- Route visualization and route-result caching.
- Google Places address autocomplete and Place Details lookup.
- Recent address-search storage in localStorage.

This separation means common UI interactions such as selecting a task, hiding a vehicle, or focusing a route do not cause repeated Supabase queries.

4. Daily Service-Task Query

The main dashboard retrieves only the records required for the selected scheduled_date rather than loading the complete mirrored operational dataset.

```
SELECT *
FROM service_tasks
WHERE scheduled_date = selected_date
ORDER BY vehicle_id, time_window;
```

Implemented optimization: composite database index

The project now includes a PostgreSQL composite index that matches the main access pattern:

```
CREATE INDEX IF NOT EXISTS
service_tasks_scheduled_date_vehicle_id_time_window_idx
ON public.service_tasks (scheduled_date, vehicle_id, time_window);
```

scheduled_date leads the index because both the daily query and recommendation query filter by date first. vehicle_id and time_window follow it because the daily dashboard sorts by those fields. This reduces unnecessary table scanning and can also avoid an additional sort for the common daily query as the historical table grows.

The index is an implemented scale optimization, not only a future recommendation. Its value becomes more significant as the mirrored operational dataset accumulates more historical records.

5. Recommendation Query

When a dispatcher searches for a new service location, getRecommendations() retrieves service tasks only for the next four days. The application then validates task coordinates, calculates Haversine distance, keeps tasks within 20 km, keeps the closest task per vehicle/date pair, and sorts the final candidates by distance.

Current advantage

The query is bounded by a short operational time window rather than the entire database. The composite scheduled_date index also supports the date-range portion of this query.

Potential bottleneck

Geographical filtering still occurs in Next.js application logic after the four-day records have been transferred from Supabase. If a future organization has thousands or tens of thousands of tasks in that window, many rows could be transferred only to be discarded after distance calculation.

Future improvement

At larger scale, candidate filtering should move closer to PostgreSQL, for example through a latitude/longitude bounding-box pre-filter or PostGIS spatial queries and a spatial index. Exact Haversine-style distance filtering can then run on a much smaller candidate set.

6. Route Optimization and Request Control

Each vehicle route is optimized independently in the browser through the Mapbox Optimization API. The implementation already includes several mechanisms that prevent unnecessary repeated work.

- Only vehicles with at least two valid coordinates require an optimization request.
- Route inputs are derived from stable vehicle coordinates, so unrelated UI state changes do not change the request key.
- An in-memory cache stores successful and failed route results for the current page lifetime.
- Requests for different vehicles run independently instead of one slow vehicle blocking every other route.
- AbortController cancels route requests that are no longer relevant after the inputs change.
- Coordinates are rounded to six decimal places in the request key, preventing insignificant floating-point differences from unnecessarily invalidating the cache.

7. Mapbox Optimization Limit

The current Mapbox Optimization API flow is explicitly limited to 12 coordinates per vehicle request. The application handles this limit instead of sending unsupported oversized requests.

- The first 12 valid service-task coordinates participate in optimization.
- Additional valid tasks remain visible as markers.
- Invalid-coordinate tasks are excluded from route calculations.

The limitation means that a vehicle with more than 12 routable stops is not fully optimized in the current version. A future large-fleet implementation could split routes geographically, make multiple optimization requests, or use a routing service designed for larger Vehicle Routing Problems.

8. Google Places Request Efficiency

Address search uses the Google Places API in the browser. The current implementation avoids sending a request on every keystroke.

- Autocomplete starts only after at least three characters are entered.
- Input is debounced by approximately 300 milliseconds.
- The current autocomplete request is cancelled when the input changes before completion.
- A Google Places session token groups autocomplete with the selected Place Details lookup.
- The five most recent successful searches are stored locally; selecting a recent search reuses its saved coordinates instead of performing another Places lookup.

At larger usage volumes, Google Places and Mapbox quotas/costs should be monitored because external API traffic grows with the number of active dispatcher browsers.

9. Avoiding Unnecessary Data Loading

1. The main dashboard loads a single selected workday.
2. Recommendations read only a four-day future date range.
3. Vehicle grouping, selection, visibility and focus are performed locally after daily data is loaded.
4. Derived route inputs and map data are memoized in the client where appropriate.
5. Recent address searches are browser-local and do not create additional database rows.

10. Pagination

Traditional pagination is intentionally not used for the main daily map. The product needs a complete geographical picture of the selected workday; paginating service tasks would produce an incomplete map and could hide operational activity from the dispatcher.

For the expected daily task volume, date scoping is the primary bounding mechanism. If daily datasets grow to thousands of tasks, a future implementation should consider marker clustering, viewport-based map loading, GeoJSON layers, virtualized sidebar lists, and pagination only for secondary tabular views where a complete map is not required.

11. Client-Side Rendering Limits

Every usable service location is currently represented by an interactive marker, and route geometries are rendered in the browser. This is appropriate for moderate daily datasets but can become expensive when hundreds or thousands of markers and multiple route lines are visible simultaneously.

For substantially larger daily datasets, the preferred improvements would be marker clustering, GeoJSON source/layer rendering instead of many React marker components, viewport-based loading, and aggregated views at lower zoom levels.

12. Database Payload Size

The current daily and recommendation queries use select('*'). This is acceptable because the dashboard consumes many of the service-task fields, but it can become wasteful if the mirrored source schema expands with many additional or large columns in the future.

At that point, each query should request only the fields needed for its specific flow. The recommendation query is a particularly strong candidate because its calculation primarily needs date, vehicle identity, coordinates, installer/time-window display fields, and task identity.

13. Concurrent Users and Shared Caching

With more concurrent dispatchers, several browsers may request the same daily tasks and independently calculate the same vehicle routes. The current route cache is local to one browser page and is not shared across dispatchers.

For higher concurrency, a shared route cache keyed by date, vehicle ID, and route coordinates could reduce duplicate Mapbox calls. Frequently requested daily data could also be cached where compatible with authentication and freshness requirements. Any such cache should be introduced only after measuring real production bottlenecks and defining how quickly changes synchronized from the upstream operational source must become visible.

14. Current Scale Limitations

1. Geographical recommendation filtering is performed in application logic after the four-day dataset is retrieved.
2. Route caching is local to a browser page, not shared across dispatchers.
3. Mapbox optimization is limited to 12 coordinates per vehicle request.
4. All markers for the selected workday are rendered in the browser at the same time.
5. Database queries currently request all service_tasks columns.
6. There is no shared application-level cache for daily task data or route results.
7. The project has not yet been validated by a production-style load test.

15. Future Improvements for Larger Scale

1. Move geographical recommendation filtering into PostgreSQL/PostGIS or apply a database-side bounding-box pre-filter.
2. Introduce shared caching for repeated route calculations when freshness requirements allow it.
3. Use marker clustering, GeoJSON layers, virtualization, or viewport-based loading for very large daily maps.
4. Adopt a routing solution that supports larger multi-stop vehicle-routing problems when vehicles regularly exceed 12 stops.
5. Select only required columns for flows whose payload can be reduced.
6. Add production monitoring for database latency, Mapbox/Google API usage, failed requests, server response times, and client rendering performance.
7. Define synchronization freshness targets and monitoring for whichever upstream ERP or operational source system supplies the mirrored dataset.
8. Run realistic load tests before increasing the expected concurrent dispatcher or daily-task count.

16. Summary

The current architecture supports basic scale by bounding database queries by date, using an implemented composite index for the primary service_tasks access pattern, keeping most interactive map operations client-side, controlling Google Places requests through minimum input length and debouncing, and caching/cancelling Mapbox route requests in the browser.

The largest future scale opportunities are database-side geographical filtering, shared route caching, more efficient rendering of very large map datasets, and a routing strategy that exceeds the current 12-stop optimization limit. Because the dashboard consumes a normalized operational mirror, these scaling strategies remain applicable regardless of whether the upstream source is SAP, another ERP, or a different organizational database. The design intentionally avoids adding distributed infrastructure before the expected workload requires it.